

FirstCry Clone E-Commerce Platform

Full System Wireframe & Workflow Architecture

Repository: C:\Personal\webdev\firstcry

1. Project Overview

FirstCry Clone is a quick-commerce/e-commerce platform with customer storefront workflows, OTP authentication, cart and checkout, order processing, payment integration, product catalog management, media upload, SignalR notifications, and an RBAC-protected admin panel. The architecture is shaped for SaaS-style scaling with a Next.js frontend and ASP.NET Core API backend.

Engineering goals include separation of concerns, reusable API communication, resilient OTP/cache fallback, admin-friendly operations, and production-readiness through observability, deployment discipline, and data integrity controls.

2. Complete System Architecture

```
+-----+
| Browser: customer storefront / admin console |
+-----+
| Next.js App Router: layouts, route groups, middleware, server/client components |
+-----+
| State: React Query server cache | Zustand auth/cart/ui/checkout stores |
+-----+
| API client: Axios, JWT attachment, retries, refresh-token replay |
+-----+
| Next.js Proxy: /api/v1/* -> ASP.NET API | /hubs/* -> SignalR |
+-----+
| ASP.NET Core: middleware -> controllers -> MediatR -> EF Core/services |
+-----+
| Data/infra: SQL Server | Redis | Cloudinary | SignalR | payment provider |
+-----+
``mermaid
graph TD
  U[User/Admin Browser] --> FE[Next.js App Router UI]
  FE --> STATE[React Query cache + Zustand stores]
  STATE --> AX[Axios client]
  AX --> INT[Interceptors: JWT, retry, refresh]
  INT --> PROXY[Next.js proxy /api and /hubs]
  PROXY --> API[ASP.NET Core API]
  API --> MW[Middleware: exception, CORS, security, logging, auth]
  MW --> CTRL[Controller v1]
  CTRL --> MED[MediatR command/query]
  MED --> EF[EF Core DbContext + repositories]
  EF --> DB[(SQL Server)]
  MED --> REDIS[(Redis OTP/cache)]
  DB --> EF --> MED --> CTRL --> API --> PROXY --> AX --> STATE --> FE
  ...
```

3. Frontend Architecture Wireframe

```
``mermaid
graph TD
  APP[app/ routes] --> LAYOUTS[Root/Admin layouts]
  APP --> PAGES[Storefront/Admin/Account pages]
  LAYOUTS --> MID[Next middleware: auth/admin guard]
  PAGES --> COMPONENTS[Shared + domain components]
  COMPONENTS --> HOOKS[React Query hooks]
  COMPONENTS --> STORES[Zustand auth/cart/ui/checkout stores]
  HOOKS --> SERVICES[services/* and lib/api/*]
  STORES --> SERVICES
  SERVICES --> AXIOS[Axios client + interceptors]
  AXIOS --> PROXY[Next.js rewrites to backend]
  ...
```

The frontend uses App Router pages/layouts, middleware route guards, shared/domain components, React Query for server-state, Zustand for client state, service files for API modules, and Axios interceptors for auth, retry, and refresh replay.

4. Backend Architecture Wireframe

```
``mermaid
graph TD
  HTTP[HTTP request] --> PIPE[ASP.NET middleware pipeline]
  PIPE --> AUTH[JWT authentication + RBAC authorization]
  AUTH --> CTRL[Controllers v1]
  CTRL --> DTO[DTO/request validation]
  DTO --> MED[MediatR]
  MED --> CQRS[Commands and Queries]
  CQRS --> REPO[Repositories / UnitOfWork]
  ...
```

```
CQRS --> SVC[Infrastructure services]
REPO --> EF[ApplicationDbContext]
EF --> SQL[(SQL Server)]
SVC --> REDIS[(Redis)]
SVC --> CLOUD[Cloudinary]
SVC --> PAY[Razorpay/mock]
SQL --> RESP[ApiResponse<T>]
...
```

The backend routes requests through middleware, JWT/RBAC authorization, controllers, DTO validation, MediatR commands/queries, repositories/services, EF Core, SQL Server, Redis, Cloudinary, and payment integrations.

5. Authentication Flow Wireframe

```

sequenceDiagram
    participant User
    participant UI as Login UI
    participant API as AuthController
    participant OTP as Redis/InMemory OTP
    participant SMS as SMS Service
    participant DB as SQL Server
    participant JWT as TokenService
    User->>UI: Enter phone
    UI->>API: POST /auth/send-otp
    API->>OTP: Generate OTP + cooldown
    API->>SMS: Send OTP or demo log
    User->>UI: Enter OTP
    UI->>API: POST /auth/verify-otp
    API->>OTP: Validate OTP
    API->>DB: Find/create user
    API->>JWT: Generate access token with role
    API->>DB: Store refresh token
    API->>UI: AuthResponse
    UI->>UI: Hydrate Zustand + cookies/localStorage

sequenceDiagram
    participant UI
    participant AX as Axios Interceptor
    participant API as AuthController
    participant DB as SQL Server
    UI->>AX: API call returns 401
    AX->>API: POST /auth/refresh-token
    API->>DB: Load user by active refresh token
    API->>DB: Revoke old token + append replacement
    API->>AX: New access/refresh token
    AX->>UI: Replay original request

```

Redis is the preferred OTP store; in-memory fallback is registered when Redis is unavailable. Role validation is enforced by backend AdminOnly policy and frontend middleware/admin UI visibility checks.

6. Admin Panel Wireframe

Sidebar	Header: search, notifications, admin profile
Dashboard	KPI cards / module toolbar
Products	Primary table/list view
Orders	Detail drawer/modal: CRUD form, status update, review action
Customers	Pagination, filters, mutation feedback
Categories	
Inventory	
Reviews	
Coupons/Banners	

Module	Page	Backend Flow	DB Interaction	React Query Lifecycle	Runtime Status
Dashboard	/admin/dashboard	GET /api/v1/admin/dashboard	Orders, Products, Users, Inventory	React Query load -> metric cards/charts	Stable, reported verified
Products	/admin/products	GET/POST/PUT/DELETE /api/v1/admin/products	Products, Images, Categories, Brands, Inventory	CRUD mutation -> invalidate product/admin queries	Stable, reported verified
Orders	/admin/orders	GET /admin/orders; PUT /admin/orders/{id}/status	Orders, OrderItems, Payments, StatusHistory	Status mutation -> invalidate orders/dashboard	Stable, reported verified
Customers	/admin/customers	GET /admin/customers; PUT /customers/{id}/block	Users, Orders	Block/unblock mutation -> refresh customers	Stable, reported verified
Categories	/admin/categories	GET/POST/PUT/DELETE /admin/categories	Categories	CRUD mutation -> refresh category lists	Implemented, needs runtime pass
Inventory	/admin/inventory	GET /admin/inventory; GET /alerts; stock update	Inventories, WarehouseInventories, Products	Stock mutation -> refresh inventory/product lists	Implemented, needs runtime pass
Reviews	/admin/reviews	GET/PATCH/DELETE /admin/reviews	ProductReviews, Products, Users	Moderation mutation -> refresh reviews	Implemented, needs runtime pass
Coupons	/admin/coupons	GET/POST/PUT/DELETE /admin/coupons	Coupons	CRUD mutation -> refresh coupon list	Implemented, needs runtime pass
Banners	/admin/banners	GET/POST/PUT/DELETE /admin/banners	Banners, Media	CRUD mutation -> refresh storefront assets	Implemented, needs runtime pass
Settings	/admin/settings	No durable settings endpoint confirmed	Future Settings table	Local admin form behavior	Partial
Storefront	Header/banner/product surfaces	Admin banners + media + products APIs	Banners, Products, ProductImages	Publishing flow through admin CRUD	Partial

7. Product Workflow

```
```mermaid
graph TD
 A[Admin Products UI] --> F[Product form validation]
 F --> S[admin-service createAdminProduct]
 S --> AX[Axios POST /api/v1/admin/products]
 AX --> P[Next.js proxy]
 P --> C[AdminController]
 C --> CMD[CreateProductCommand]
 CMD --> EF[EF Core]
 EF --> SQL[(Products/ProductImages/Inventory)]
 SQL --> RESP[AdminProductDto]
 RESP --> RQ[React Query mutation success]
 RQ --> INV[Invalidate admin products/dashboard]
 INV --> UI[UI refresh]
```
```

Product browsing uses public catalog APIs. Product search uses `/api/v1/search`. Inventory updates flow through admin stock endpoints and invalidate relevant React Query caches after mutation success.

8. Cart & Checkout Workflow

```
```mermaid
graph TD
 PDP[Product card/detail] --> ADD[Add to cart]
 ADD --> Z[Zustand cart optimistic state]
 Z --> API[POST /cart/items]
 API --> DB[(Cart + CartItems)]
 DB --> CART[Cart response]
 CART --> CHECK[Checkout]
 CHECK --> ORDER[POST /orders]
 ORDER --> ODB[(Orders + OrderItems + StatusHistory)]
 ODB --> PAY[Payment create/verify]
 PAY --> DONE[Order confirmation + history]
```
```

Zustand provides optimistic cart state; backend cart endpoints persist authenticated state. Guest carts merge after OTP login. Checkout creates orders, initiates/validates payment, and returns confirmation/order history.

9. Order Management Workflow

```
```mermaid
graph TD
 USER[User checkout] --> PLACE[PlaceOrderCommand]
 PLACE --> ORDER[(Orders)]
 PLACE --> ITEMS[(OrderItems)]
 PLACE --> HIST[(StatusHistory)]
 ORDER --> PAY[Payment lifecycle]
 ADMIN[Admin Orders UI] --> STATUS[Update status]
 STATUS --> API[PUT /admin/orders/{id}/status]
 API --> HIST
 API --> NOTIFY[SignalR notification]
 NOTIFY --> USER
```
```

Order processing creates orders, order items, payment records, and status history. Admin updates append timeline events and can notify customers through SignalR. Inventory reduction and audit logging should be production-hardened.

10. Database Architecture

```

mermaid
erDiagram
    USERS ||--o{ ORDERS : places
    USERS ||--o{ CART : owns
    USERS ||--o{ REVIEWS : writes
    USERS ||--o{ NOTIFICATIONS : receives
    PRODUCTS ||--o{ ORDERITEMS : included_in
    PRODUCTS ||--o{ CARTITEMS : added_to
    PRODUCTS ||--o{ REVIEWS : reviewed_by
    CATEGORIES ||--o{ PRODUCTS : groups
    BRANDS ||--o{ PRODUCTS : owns
    ORDERS ||--o{ ORDERITEMS : contains
    ORDERS ||--o{ STATUSHISTORY : tracks
    ORDERS ||--|| PAYMENTS : paid_by

```

| Table | Key Fields | Relationships |
|---------------|--|--|
| Users | Id, PhoneNumber, Name, Email, Role, IsGuest, ProfileCompleted, IsDeleted | 1:N Addresses, WishlistItems, Reviews, Orders, Notifications; owns RefreshTokens |
| Roles | Role represented by User.Role plus JWT role claim | AdminOnly policy uses role claim; no separate role table confirmed |
| Products | Id, Slug, Name, Price, StockQuantity, CategoryId, BrandId, IsActive, IsDeleted | N:1 Category, N:1 Brand, 1:N ProductImages, Reviews, CartItems, OrderItems |
| Categories | Id, Name, Slug, ParentCategoryId, DisplayOrder, IsActive | Self hierarchy, 1:N Products |
| Orders | Id, OrderNumber, UserId, Status, totals, ShippingAddress, CreatedAt | 1:N OrderItems, 1:1 Payment, 1:N StatusHistory |
| OrderItems | OrderId, ProductId, Quantity, UnitPrice, Total | N:1 Order, N:1 Product snapshot data |
| Cart | Id, UserId, CreatedAt, UpdatedAt | 1:N CartItems, N:1 User |
| CartItems | CartId, ProductId, Quantity | N:1 Cart, N:1 Product |
| Reviews | UserId, ProductId, Rating, Title, Comment, Status | N:1 User, N:1 Product |
| Coupons | Code, DiscountType, Amount, Validity, UsageLimit, IsActive | Used by checkout/promotion rules |
| Notifications | UserId, Title, Message, Type, IsRead, CreatedAt | N:1 User, delivered through SignalR |
| Banners | Title, ImageUrl, TargetUrl, DisplayOrder, IsActive | Admin-managed storefront content |

11. API Communication Flow

Frontend request -> Axios -> interceptors -> JWT attachment -> Next.js proxy -> backend middleware -> authorization -> controller -> MediatR handler -> database/service -> ApiResponse<T> -> frontend unwrap/error handling -> React Query/Zustand -> render. Retry logic handles transient network errors; 401 responses can trigger refresh-token replay.

```

```mermaid
graph TD
 U[User/Admin Browser] --> FE[Next.js App Router UI]
 FE --> STATE[React Query cache + Zustand stores]
 STATE --> AX[Axios client]
 AX --> INT[Interceptors: JWT, retry, refresh]
 INT --> PROXY[Next.js proxy /api and /hubs]
 PROXY --> API[ASP.NET Core API]
 API --> MW[Middleware: exception, CORS, security, logging, auth]
 MW --> CTRL[Controller v1]
 CTRL --> MED[MediatR command/query]
 MED --> EF[EF Core DbContext + repositories]
 EF --> DB[(SQL Server)]
 MED --> REDIS[(Redis OTP/cache)]
 DB --> EF --> MED --> CTRL --> API --> PROXY --> AX --> STATE --> FE
```
```

Workflow Interaction Matrix

| Workflow | Trigger | System Path | Result |
|-----------------------|----------------------------------|--|---|
| Product browse | Customer opens catalog or search | productsApi/search -> public controllers -> MediatR queries -> EF Core | Product grid/detail, cache populated |
| Product create | Admin submits product form | admin-service -> AdminController -> CreateProductCommand -> EF Core insert | Admin table refreshed, product available to storefront |
| Inventory update | Admin adjusts stock | PUT stock endpoint -> command/service -> product/inventory tables | Stock count refreshed and low-stock alerts recalculated |
| Add to cart | Customer clicks add-to-cart | Zustand optimistic update -> POST /cart/items -> CartService/EF | Cart drawer/page reflects persisted cart |
| Guest cart merge | Guest logs in with OTP | auth-store verifies OTP -> cart.merge -> CartController merge endpoint | Guest items attached to authenticated user |
| Checkout | User confirms address/payment | POST /orders -> order handler -> payment service -> status history | Order confirmation and order history updated |
| Admin order status | Admin changes fulfillment status | PUT /admin/orders/{id}/status -> status handler -> history append | Admin table refresh and optional notification |
| Token refresh | Access token expires | Axios 401 interceptor -> /auth/refresh-token -> rotate refresh token | Original request replayed without redirect loop |
| Realtime notification | Order/delivery event emitted | API service -> SignalR hub -> Next /hubs proxy -> browser hook | Notification store receives event |

State Management Architecture

| Layer | State / Responsibility | Used By |
|--------------------------------|---|--|
| Zustand auth-store | User, token, refresh token, hydration flag, role helpers | Login, admin guard, logout, token refresh |
| Zustand cart-store | Items, totals, guest cart, optimistic mutations | Cart drawer/page, checkout, merge after login |
| Zustand checkout-store | Selected address, payment mode, checkout progress | Checkout pages and order creation |
| Zustand UI/notification stores | Drawer state, toasts, realtime notifications | Global UX feedback and SignalR events |
| React Query | Server-state cache, loading/error status, mutation invalidation | Admin lists, dashboard, products, orders, categories |
| Axios interceptors | JWT attachment, retry policy, 401 refresh replay | All protected API calls and session recovery |

12. Real-Time & Infrastructure Flow

```
```mermaid
graph LR
 API[ASP.NET API] --> HUB[SignalR NotificationHub]
 HUB --> PROXY[Next.js /hubs proxy]
 PROXY --> CLIENT[Browser SignalR client]
 API --> REDIS[(Redis cache/OTP/backplane when available)]
 API --> MEM[(In-memory fallback)]
 API --> SQL[(SQL Server)]
 API --> CLOUD[Cloudinary]
```
```

SignalR handles notification delivery; Redis supports OTP/cache/backplane when available; Cloudinary stores product media; SQL Server remains the durable source of truth.

13. Debugging & Runtime Audit Flow

```
```mermaid
graph TD
 BUG[Runtime issue] --> LOGS[Serilog + request logs]
 BUG --> BROWSER[Browser console/network]
 BUG --> API[Controller response + ApiResponse shape]
 BUG --> SQL[Migration/schema check]
 LOGS --> TRACE[Trace request id/module]
 BROWSER --> TRACE
 API --> TRACE
 SQL --> FIX[Patch DTO/schema/handler]
 TRACE --> FIX
 FIX --> VERIFY[Build + endpoint + browser verification]
```
```

| Runtime Layer | Audit Surface | Verification Method |
|-----------------|--|--|
| Browser | Network tab, console, hydration state, route redirects | Verify proxy path, auth cookie/token, React Query state, console errors |
| Next.js proxy | Rewrites `/api:/path*` and `/hubs:/path*` | Confirm backend target, status code, CORS/cookie behavior |
| API middleware | Exception/security/request logging/auth/rate limiting | Check status code, auth challenge, policy failure, trace logs |
| Controller | Route binding and DTO input | Confirm endpoint path, payload shape, ApiResponse wrapper |
| MediatR handler | Command/query business execution | Confirm validation, repository/service calls, transaction boundary |
| EF Core | DbContext query/command | Check SQL schema, migrations, includes, soft delete filter, FK constraints |
| Infrastructure | Redis, Cloudinary, SignalR, payment provider | Validate fallback, secrets, timeout behavior, external response shape |

14. Deployment Architecture

```

```mermaid
graph TD
 CLIENT[Client browser] --> CDN[CDN/static asset edge]
 CDN --> NEXT[Next.js frontend]
 NEXT --> RP[Reverse proxy / load balancer]
 RP --> API[ASP.NET Core API replicas]
 API --> REDIS[Redis cache + OTP + SignalR backplane]]
 API --> SQL[(SQL Server primary)]
 API --> CLOUD[Cloudinary media]
 API --> OBS[Logs/metrics/traces]
 SQL --> BACKUP[Automated backups]
```
```

Production deployment should use separate environments, CDN-backed frontend delivery, reverse proxy/load balancing for API replicas, Redis for shared cache/backplane, SQL backups, Cloudinary media delivery, and centralized logs/metrics/traces.

15. Current Project Progress

| Area | Completion | Runtime-Stable Surface | Pending Work |
|----------------|------------|--|--|
| Frontend | 78% | App Router pages, admin layouts, auth/cart stores, proxy API calls | Hydration consistency, notification UI, admin settings/storefront polish |
| Backend | 84% | Controllers, MediatR handlers, EF Core, JWT, RBAC, middleware | Transactions, index coverage, audit logging, production secrets |
| Authentication | 86% | OTP, Redis/in-memory fallback, JWT, refresh rotation, admin role checks | CSRF strategy, token storage hardening, production lockout policy |
| Admin Panel | 76% | Dashboard, products, orders, customers, categories, reviews, inventory, coupons, banners | Runtime pass for every CRUD module and settings persistence |
| Database | 82% | Entities, migrations, soft delete, order/cart/product/review schemas | Indexes, concurrency tokens, backup/restore strategy |
| Infrastructure | 68% | Docker, SQL Server, Redis, SignalR, Cloudinary, Next proxy, health | Monitoring, alerting, CDN, deployment pipeline |

Verified / Audited Runtime Matrix

| Module | Audited Surface | Status | Next Verification |
|-----------------|----------------------|----------------------------------|--------------------------------------|
| Admin dashboard | Frontend + backend | Reported verified | Add browser smoke test |
| Admin products | Frontend + backend | Reported verified | Add create/edit/delete e2e test |
| Admin orders | Frontend + backend | Reported verified | Add status transition test |
| Admin customers | Frontend + backend | Reported verified | Add block/unblock assertion |
| OTP auth | Code verified | Needs rerun | Verify send, verify, refresh, logout |
| Categories | Code verified | Needs runtime pass | Check public + admin CRUD |
| Cart | Code verified | Needs authenticated runtime pass | Check add/update/remove/merge |
| Checkout/orders | Code verified | Needs full flow pass | Check order + payment + history |
| SignalR | Hook and hub present | Needs live socket test | Check order notification path |

Engineering Conclusion

The platform has a strong architecture base: App Router frontend, Zustand and React Query state layers, Axios proxy communication, ASP.NET Core controller/CQRS backend, EF Core persistence, Redis fallback strategy, JWT/RBAC authorization, and admin workflows. Production readiness now depends on transaction safety, indexes, observability, secrets, webhook enforcement, and e2e verification.